

```
int float bool char str
```

Typy danych

Dział I · Typy danych w projektowaniu aplikacji

Projektowanie oprogramowania

Technik programista · klasa II · INF.04.3.1

Opracował: Bartosz Bryniarski

Plan działu

Pięć lekcji plus zajęcia ćwiczeniowe

1 Rola typów danych w projekcie

Typowanie statyczne i dynamiczne

2 Typy liczbowe stałoprzecinkowe

Zakresy, przepełnienie

3 Typy zmiennoprzecinkowe

Precyzja i błędy zaokrągleń

4 Typ logiczny i znakowy

Kodowanie znaków: ASCII, Unicode, UTF-8

5 Typ łańcuchowy

Operacje i reprezentacja w pamięci

6 Ćwiczenia

Dobór typu prostego do problemu programistycznego

1

Rola typów danych w projekcie

Typowanie statyczne i dynamiczne

po co typ · typy proste i złożone · statyczne vs dynamiczne · konwersje

Po co komputerowi typ danych?

Pamięć zna tylko zera i jedyńki

```
01000001 01000001 01000001 01000001
```

```
jako liczba całkowita -> 1094795585
```

```
jako cztery znaki -> AAAA
```

```
jako liczba float -> 12,078431
```

Ten sam ciąg bitów. To typ danych mówi procesorowi, jak odczytać zawartość pamięci. Bez typu bity nic nie znaczą.

Typ danych = umowa o znaczeniu bitów

Co określa typ danych

Trzy rzeczy naraz

Zbiór wartości

Jakie wartości można zapisać. Typ `int` nie przyjmie tekstu, a `bool` tylko dwie wartości.

Zbiór operacji

Co wolno na tych danych zrobić. Liczby można dzielić, napisy łączyć, ale nie odwrotnie.

Rozmiar w pamięci

Ile bajtów zajmuje jedna wartość i jak są w nich ułożone bity.

Konsekwencja projektowa: dobór typu to jednocześnie decyzja o zakresie danych, dopuszczalnych operacjach i zużyciu pamięci. Zmiana typu w gotowej aplikacji jest kosztowna.

Typy proste i złożone

Podstawowy podział

Typy proste (prymitywne)

- Przechowują jedną wartość
- Wbudowane w język
- Stały, znany rozmiar
- int, float, bool, char

Typy złożone (strukturalne)

- Zbudowane z typów prostszych
- Zmienny rozmiar
- Własne operacje
- tablica, rekord, lista, obiekt

W tym dziale zajmujemy się typami prostymi. Typy złożone to temat kolejnego działu.

Typowanie statyczne i dynamiczne

Kiedy sprawdzany jest typ

Statyczne – podczas kompilacji

```
// C++, Java, C#  
int wiek = 18;  
wiek = "osiemnascie";  
// blad juz przy kompilacji
```

Statyczne

Błędy wcześniej, szybszy kod, więcej pisania

Dynamiczne – podczas działania

```
# Python, JavaScript, PHP  
wiek = 18  
wiek = "osiemnascie"  
# poprawne - typ ma wartosc
```

Dynamiczne

Szybkie prototypowanie, błędy dopiero u użytkownika

Typowanie silne i słabe

Jak bardzo język pilnuje typów

Python (silne)

```
"5" + 3    -> TypeError
```

JavaScript (słabe)

```
"5" + 3    -> "53"
```

```
"5" - 3     -> 2
```

```
[] + {}    -> "[object Object]"
```

Typowanie silne

Język nie zgadnie za Ciebie. Konwersję trzeba napisać wprost: `int("5") + 3`

Typowanie słabe

Język milcząco konwertuje typy. Wygodne, dopóki nie zwróci bzdury bez ostrzeżenia.

To dwie niezależne osie: Python jest dynamiczny i silny, C jest statyczny i raczej słaby.

Deklaracja i wnioskowanie typu

Skąd kompilator wie, jaki to typ

```
// deklaracja jawna
int liczba = 42;           // C++
String imie = "Ala";       // Java

// wnioskowanie typu (inferencja)
auto liczba = 42;          // C++11  -> int
var imie = "Ala";          // C#      -> string
let x = 3.14;              // TypeScript -> number
```

Uwaga: wnioskowanie typu to nadal typowanie statyczne. Typ zostaje ustalony raz, przy kompilacji, i już się nie zmieni.

Konwersja typów

Rzutowanie jawne i niejawne

Niejawna (automatyczna)

```
int a = 5;  
double b = a;    // 5.0  
// rozszerzająca - bezpieczna
```

Jawna (rzutowanie)

```
double b = 5.99;  
int a = (int) b;    // 5  
// zawężająca - strata danych
```

Konwersja zawężająca zawsze coś traci. Rzutowanie double na int obcina część ułamkową (nie zaokrągla!), a rzutowanie int na short może zmienić wartość nie do poznania.

```
(int) 5.99    -> 5           (int) -5.99    -> -5
```

Dlaczego to jest decyzja projektowa

Konsekwencje złego doboru typu

Za mały zakres

Przepełnienie i błędne wyniki, często ujawniane dopiero po miesiącach

Za duży zakres

Marnowanie pamięci – istotne w bazach danych i systemach wbudowanych

Zły rodzaj typu

float do kwot pieniężnych = różnice w rozliczeniach

Brak spójności

Ten sam PESEL raz jako liczba, raz jako tekst – błędy przy integracji

Typ dobiera się na etapie projektowania, razem z projektem struktury danych i bazy. Zmiana typu kolumny w działającym systemie oznacza migrację danych i przegląd całego kodu.

Temat 1 – podsumowanie

Co trzeba umieć

- Typ danych określa zbiór wartości, zbiór operacji i rozmiar w pamięci
- Typy dzielimy na proste (jedna wartość) i złożone (zbudowane z prostszych)
- Typowanie statyczne sprawdza typy przy kompilacji, dynamiczne w czasie działania
- Typowanie silne i słabe to osobna oś – mówi, jak chętnie język konwertuje typy
- Wnioskowanie typu (auto, var) to nadal typowanie statyczne
- Konwersja zawężająca traci dane – rzutowanie na int obcina, nie zaokrągla
- Dobór typu jest decyzją projektową, nie techniczną drobnostką

2

Typy liczbowe stałoprzecinkowe

Zakresy i przepelnienie

bit i bajt · kod U2 · zakresy typów · overflow

Liczba w pamięci komputera

Bit, bajt, system dwójkowy

1 bajt = 8 bitów

0 0 1 0 1 1 0 1

128 64 32 16 8 4 2 1 <- wagi pozycji

32 + 8 + 4 + 1 = 45

8 bitów

256 kombinacji

16 bitów

65 536 kombinacji

32 bity

ponad 4 miliardy

n bitów daje dokładnie 2^n różnych wartości – ani jednej więcej.

Liczby bez znaku i ze znakiem

Ten sam bajt, dwa różne zakresy

unsigned – bez znaku

wszystkie 8 bitów = wartość

zakres: 0 .. 255

wzor: 0 .. $2^n - 1$

signed – ze znakiem (kod U2)

najstarszy bit = znak

zakres: -128 .. 127

wzor: $-2^{(n-1)}$.. $2^{(n-1)} - 1$

Kod uzupełnień do dwóch (U2). Dzięki niemu procesor odejmuje tym samym układem, którym dodaje. Zakres ujemny jest o jeden szerszy, bo zero zajmuje jedno miejsce po stronie dodatniej.

Zakresy typów całkowitych

Wartości, które trzeba znać

Typ	Bity	Zakres ze znakiem	Zakres bez znaku	Typowe użycie
<code>byte</code> / <code>int8</code>	8	-128 .. 127	0 .. 255	bajt danych, kolor RGB
<code>short</code> / <code>int16</code>	16	-32 768 .. 32 767	0 .. 65 535	rok, kod pocztowy
<code>int</code> / <code>int32</code>	32	±2,1 mld	0 .. 4,29 mld	domyślny typ całkowity
<code>long</code> / <code>int64</code>	64	±9,22 · 10 ¹⁸	0 .. 1,84 · 10 ¹⁹	identyfikatory, czas

Do zapamiętania: `int` (32 bity) mieści około 2,1 miliarda ze znakiem. To najczęstsza granica, o którą rozbijają się aplikacje.

W Javie typy całkowite są zawsze ze znakiem. W C/C++ rozmiar `int` zależy od platformy.

Wyjątek: Python

Liczby całkowite bez limitu

```
>>> 2 ** 200
1606938044258990275541962092341162602522202993782792835301376

>>> import sys
>>> sys.getsizeof(2 ** 200)
104          # bajtow - obiekt rośnie razem z liczbą
```

Zaleta

Brak przepełnienia. Nie trzeba dobierać rozmiaru typu.

Koszt

Wolniejsze obliczenia i większe zużycie pamięci niż w C czy Javie.

Uwaga na egzaminie: pytania o zakresy dotyczą typów o stałym rozmiarze, nie Pythona.

Przepełnienie (overflow)

Co się dzieje po przekroczeniu zakresu

```
byte licznik = 127;      // maksimum dla int8
licznik = licznik + 1;

wynik:  -128              // zamiast 128

01111111  =  127
10000000  = -128      <- przeniesienie na bit znaku
```

Zawijanie (wraparound). Licznik przeskakuje z maksimum na minimum. W większości języków dzieje się to cicho – bez błędu i bez ostrzeżenia.

Przepełnienie w praktyce

Trzy znane przypadki

Ariane 5 (1996)

Rzutowanie 64-bitowej liczby zmiennoprzecinkowej na 16-bitową całkowitą. Rakieta uległa zniszczeniu 37 sekund po starcie. Straty: około 370 mln dolarów.

YouTube (2014)

Licznik odtworzeń był typem int32 o zakresie do 2,1 mld. Popularny teledysk zbliżył się do limitu i licznik trzeba było przebudować na int64.

Problem roku 2038

Czas uniksowy liczony jako int32 sekund od 1970 roku przepełni się 19 stycznia 2038. Systemy 32-bitowe cofną się do roku 1901.

Wspólny mianownik: typ dobrany pod dzisiejsze dane, bez zapasu na wzrost. Przy projektowaniu warto pytać, jak duża będzie ta wartość za dziesięć lat.

Dzielenie liczb całkowitych

Pułapka, która nie rzuca błędu

```
int a = 7, b = 2;  
int wynik = a / b;           // 3    - część ułamkowa znika  
double zły  = a / b;         // 3.0  - dzielenie już się odbyło  
double dobry = (double) a / b; // 3.5  
  
7 % 2  -> 1                  // reszta z dzielenia (modulo)
```

Zastosowanie div

Podział na strony, przeliczanie jednostek, indeksy tablic

Zastosowanie mod

Parzystość, cykliczność, cyfra kontrolna, reszta czasu

Jak dobrać typ całkowity

Prosty algorytm decyzyjny

1 Czy wartość może być ujemna?

Nie → typ bez znaku daje dwa razy większy zakres dodatni

2 Jaka jest maksymalna wartość?

Oszacuj ją dla najgorszego przypadku, nie dla dzisiejszych danych

3 Dodaj zapas

Dane rosną. Identyfikatory rekordów prawie zawsze zasługują na 64 bity

4 Sprawdź kontekst

Baza danych, API i kod muszą używać tego samego zakresu

Temat 2 – podsumowanie

Co trzeba umieć

- n bitów daje 2^n różnych wartości
- Bez znaku: 0 do 2^n-1 . Ze znakiem (U2): -2^{n-1} do $2^{n-1}-1$
- int8: -128..127 · int16: ± 32 tys. · int32: $\pm 2,1$ mld · int64: $\pm 9,2$ tryliona
- Przepelnienie zawija wartość z maksimum na minimum, zwykle bez żadnego błędu
- Python ma liczby całkowite o dowolnej precyzji – kosztem wydajności
- Dzielenie dwóch liczb całkowitych daje liczbę całkowitą, część ułamkowa przepada
- Typ dobieramy z zapasem, dla najgorszego przypadku

3

Typy zmiennoprzecinkowe

Precyzja i błędy zaokrągleń

IEEE 754 · $0.1 + 0.2$ · NaN i nieskończoność · pieniądze

Po co liczby zmiennoprzecinkowe

Bardzo małe i bardzo duże naraz

```
masa elektronu      0,00000000000000000000000000000091 kg
predkosc swiatla    299 792 458 m/s
```

```
zapis naukowy:      9,1 x 10^-31
                    2,998 x 10^8
```

```
znak | mantysa | wykladnik (cecha)
```

Przecinek jest ruchomy. Ta sama liczba bitów obsługuje zarówno wartości mikroskopijne, jak i astronomiczne – kosztem tego, że nie każdą wartość da się zapisać dokładnie.

Standard IEEE 754

Budowa liczby zmiennoprzecinkowej

Typ	Bity	Znak	Cecha	Mantysa	Cyfry znaczące
float	32	1	8	23	około 7
double	64	1	11	52	około 15–16

$$\text{wartosc} = (-1)^{\text{znak}} \times 1,\text{mantysa} \times 2^{(\text{cecha} - \text{bias})}$$

float

Zakres do $\pm 3,4 \cdot 10^{38}$. Wystarcza do grafiki i pomiarów.

double

Zakres do $\pm 1,8 \cdot 10^{308}$. Domyślny typ w Pythonie i JavaScriptcie.

Najsłynniejszy błąd w programowaniu

0.1 + 0.2 nie równa się 0.3

```
>>> 0.1 + 0.2
0.30000000000000004

>>> 0.1 + 0.2 == 0.3
False

>>> print(f"{0.1:.20f}")
0.100000000000000000555
```

To nie jest błąd Pythona. Ten sam wynik daje Java, C++, JavaScript i arkusz kalkulacyjny. Wynika ze standardu IEEE 754, a nie z języka.

Dlaczego tak się dzieje

0,1 nie ma skończonego zapisu dwójkowego

```
system dziesiętny: 1/3 = 0,333333... (nieskończone)
system dwójkowy:   1/10 = 0,000110011001100... (nieskończone)
```

```
0,5    = 2^-1          -> zapisywalne dokładnie
0,25   = 2^-2          -> zapisywalne dokładnie
0,75   = 2^-1 + 2^-2   -> zapisywalne dokładnie
0,1    = ?             -> NIE do zapisania
```

Zasada: dokładnie zapisać można tylko ułamki, których mianownik jest potęgą dwójki. Wszystko inne zostaje zaokrąglone do najbliższej dostępnej wartości.

Jak porównywać liczby zmiennoprzecinkowe

Nigdy operatorem równości

```
# ZLE
if a == b:

# DOBRZE - porównanie z tolerancja
EPSILON = 1e-9
if abs(a - b) < EPSILON:

# w Pythonie gotowa funkcja:
math.isclose(0.1 + 0.2, 0.3)    ->  True
```

Epsilon dobiera się do skali danych. Przy kwotach w złotych wystarczy 0,001, przy obliczeniach fizycznych trzeba znacznie mniej.

Wartości specjalne

Nieskończoność i NaN

Wartość	Powstaje przez	Zachowanie
inf	1.0 / 0.0 lub przekroczenie zakresu	inf + 5 nadal daje inf
-inf	-1.0 / 0.0	porównywalna, mniejsza od wszystkiego
nan	0.0 / 0.0, sqrt(-1), inf - inf	nan != nan – nawet sama ze sobą

```
>>> float('nan') == float('nan')
False                                # jedyna wartosc rozna od samej siebie
>>> math.isnan(x)                   # jedyny poprawny sposob sprawdzenia
```

NaN w wyniku obliczeń oznacza, że gdzieś wcześniej pojawiła się operacja niedozwolona.

Kumulacja błędów

Mały błąd powtórzony milion razy

```
suma = 0.0
for i in range(1_000_000):
    suma += 0.1

oczekiwane: 100000.0
otrzymane: 100000.00000133288
```

Sumowanie w pętli

Każde dodanie dokłada odrobinę błędu. Przy dużej liczbie iteracji błąd staje się widoczny.

Odejmowanie bliskich liczb

Utrata cyfr znaczących. Odjęcie 1,0000001 od 1,0000002 zostawia bardzo mało informacji.

Pieniądze nigdy na float

Reguła bez wyjątków

Źle

```
cena = 19.99
suma = cena * 3
# 59.970000000000001
```

Typ dziesiętny

Decimal w Pythonie, BigDecimal w Javie,
DECIMAL/NUMERIC w bazie danych

Dobrze

```
from decimal import Decimal
cena = Decimal('19.99')
# 59.97 dokładnie
```

Liczby całkowite groszy

Zamiast 19,99 zł przechowujemy 1999 groszy jako int.
Proste i szybkie.

Kolumna kwotowa w bazie danych to DECIMAL(10,2), nigdy FLOAT.

Patriot, 1991

Kiedy błąd zaokrąglenia kosztuje życie

Mechanizm

Zegar systemowy liczył dziesiąte części sekundy. Wartość 0,1 zapisana na 24 bitach miała mikroskopijny błąd.

Kumulacja

Po 100 godzinach nieprzerwanej pracy błąd zegara urósł do około jednej trzeciej sekundy.

Skutek

Przy prędkości pocisku około 1,7 km/s dało to błąd namiaru rzędu 600 metrów. Cel nie został przechwycony.

Wniosek projektowy: system testowano w krótkich cyklach, więc błąd nigdy nie zdążył się ujawnić. Warunki testów muszą odpowiadać rzeczywistemu użyciu.

Temat 3 – podsumowanie

Co trzeba umieć

- Liczba zmiennoprzecinkowa składa się ze znaku, cechy i mantysy (standard IEEE 754)
- float daje około 7 cyfr znaczących, double około 15–16
- Dokładnie zapisywalne są tylko ułamki o mianowniku będącym potęgą dwójki
- $0.1 + 0.2$ nie równa się 0.3 – to cecha standardu, nie błąd języka
- Liczby zmiennoprzecinkowe porównujemy z tolerancją, nigdy operatorem `==`
- Wartości specjalne: `inf`, `-inf` oraz `nan`, która nie równa się samej sobie
- Do kwot pieniężnych używamy typu dziesiętnego albo liczb całkowitych groszy

4

Typ logiczny i znakowy

Kodowanie znaków: ASCII, Unicode, UTF-8

bool · tablice prawdy · ASCII · UTF-8

Typ logiczny

Dwie wartości, jedna decyzja

```
bool czyPelnoletni = true;      // C++, Java  
czy_pelnoletni = True          # Python
```

rozmiar: 1 bit informacji, ale zwykle 1 bajt w pamięci
(procesor adresuje bajty, nie pojedyncze bity)

Skąd się biorą

Wynik porównania ($a > b$), wynik funkcji sprawdzającej, flaga stanu

Do czego służą

Warunki if, warunki pętli, flagi konfiguracji, kolumny TAK/NIE w bazie

Nazwa od George'a Boole'a, matematyka, który w XIX wieku sformalizował algebrę logiki.

Operatory logiczne

Tablica prawdy

a	b	a AND b	a OR b	NOT a	a XOR b
fałsz	fałsz	fałsz	fałsz	prawda	fałsz
fałsz	prawda	fałsz	prawda	prawda	prawda
prawda	fałsz	fałsz	prawda	fałsz	prawda
prawda	prawda	prawda	prawda	fałsz	fałsz

AND (&&, and)

Prawda tylko wtedy, gdy oba warunki są prawdziwe

OR (||, or)

Prawda, gdy choć jeden warunek jest prawdziwy

NOT (!, not)

Odwraca wartość logiczną

Skrócone obliczanie warunków

Short-circuit evaluation

```
if (lista != null && lista.size() > 0)
    // jesli lewa strona jest falszem,
    // prawa NIE zostanie w ogole obliczona

if (uzytkownik == null || uzytkownik.czyAdmin())
    // jesli lewa strona jest prawda, prawa sie pomija
```

Bezpieczeństwo

Chroni przed odwołaniem do wartości pustej. Kolejność warunków ma znaczenie.

Wydajność

Kosztowne sprawdzenia (zapytanie do bazy) warto stawiać jako ostatnie.

Zamiana kolejności warunków w tym zapisie może wywołać błąd wykonania.

Wartości prawdziwe i fałszywe

Truthy i falsy w językach dynamicznych

```
# Python - te wartosci sa traktowane jak fałsz:  
False    None    0    0.0    ""    []    {}    set()  
  
if lista:           # zamiast if len(lista) > 0  
    print("cos jest")
```

Wygodne, ale ryzykowne. Wartość 0 i pusty tekst są tu nie do odróżnienia od braku danych. Gdy zależy nam na rozróżnieniu, trzeba pisać warunek wprost: `if x is not None`.

Języki statyczne

Java i C# wymagają wyrażenia typu `bool`. Liczba w warunku nie przejdzie kompilacji.

Języki dynamiczne

Python, JavaScript i PHP konwertują każdą wartość na logiczną.

Typ znakowy

Znak to liczba

```
char litera = 'A';  
int kod = (int) litera;    // 65  
  
'A' = 65      'a' = 97      roznica = 32  
'0' = 48      ' ' = 32  
  
char nastepna = (char) ('A' + 1);    // 'B'
```

Znak jest przechowywany jako liczba. Dlatego znaki można porównywać i sortować, a zamiana wielkiej litery na małą sprowadza się do dodania 32.

Uwaga: apostrofy oznaczają znak, cudzysłowy tekst. 'A' to nie to samo co "A".

ASCII

Pierwszy standard kodowania

7 bitów

128 kodów: 0–127

Rok 1963

Standard amerykański

Bez ogonków

Tylko alfabet łaciński

Zakres kodów	Zawartość	Przykłady
0–31	Znaki sterujące	9 = tabulator, 10 = nowa linia
32–47	Spacja i interpunkcja	32 = spacja, 33 = wykrzyknik
48–57	Cyfry	48 = '0', 57 = '9'
65–90	Wielkie litery	65 = 'A', 90 = 'Z'
97–122	Małe litery	97 = 'a', 122 = 'z'

Cyfry mają kody od 48 – dlatego kod znaku minus 48 daje wartość cyfry.

Problem polskich znaków

Epoka stron kodowych

```
ASCII ma 128 kodów - brak miejsca na a c e l n o s z z
```

```
rozwiązanie: użyć osiemnastego bitu -> kody 128-255
```

```
ISO-8859-2 (Latin-2)    Windows-1250    Mazovia    CP852  
ten sam bajt = inna litera w każdym z tych standardów
```

Skutek: plik zapisany w jednym standardzie, a otwarty w innym, wyświetla przypadkowe znaki. To zjawisko nazywa się mojibake, a napis w rodzaju żółć gęśla żąń zamienia się w ciąg krzaków.

Problem dotyczył też innych języków. Każdy region miał własną stronę kodową i własne kłopoty.

Unicode

Jeden numer dla każdego znaku świata

```
U+0041    ->  A
U+0104    ->  A z ogonkiem
U+20AC    ->  znak euro
U+1F600   ->  emotikona uśmiechu
```

ponad 150 tysięcy znaków, alfabety wszystkich języków,
znaki matematyczne, historyczne pisma, emoji

Kluczowe rozróżnienie: Unicode przypisuje znakom numery (punkty kodowe). Nie mówi, jak zapisać je w bajtach – od tego są kodowania UTF.

UTF-8, UTF-16, UTF-32

Trzy sposoby zapisu tych samych numerów

Kodowanie	Bajty na znak	Zgodność z ASCII	Zastosowanie
UTF-8	1 do 4, zmienna	Pełna	Internet, pliki, bazy danych
UTF-16	2 lub 4	Brak	Wewnętrznie w Javie i Windows
UTF-32	zawsze 4	Brak	Rzadko – marnuje pamięć

```
"A"      -> 1 bajt   (41)
"a z ogonkiem" -> 2 bajty (C4 85)
"emoji"   -> 4 bajty (F0 9F 98 80)
```

UTF-8 to standard domyślny. Ponad 98% stron internetowych używa właśnie tego kodowania.

Kodowanie w praktyce

Gdzie to ustawiamy

```
<meta charset="UTF-8">                                <!-- HTML -->

open('plik.txt', encoding='utf-8')                    # Python

CREATE DATABASE sklep                                  -- MySQL
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_polish_ci;
```

Zasada jednego kodowania

Edytor, plik źródłowy, baza danych i nagłówki HTTP muszą mówić tym samym językiem.

MySQL: utf8mb4

Starsze utf8 w MySQL obsługuje tylko 3 bajty i gubi emoji. Poprawna nazwa to utf8mb4.

Temat 4 – podsumowanie

Co trzeba umieć

- Typ logiczny przechowuje dwie wartości; w pamięci zajmuje zwykle jeden bajt
- Operatory AND, OR, NOT i XOR – trzeba umieć wypełnić tablicę prawdy
- Skrócone obliczanie pomija drugi warunek, gdy wynik jest już przesądzony
- Znak jest przechowywany jako liczba: 'A' = 65, 'a' = 97, '0' = 48
- ASCII to 7 bitów i 128 kodów – bez polskich znaków
- Unicode nadaje numery znakom, UTF-8 określa sposób zapisania ich w bajtach
- UTF-8 jest zmiennej długości (1–4 bajty) i zgodne z ASCII

5

Typ łańcuchowy

Operacje i reprezentacja w pamięci

budowa napisu · niemutowalność · operacje · koszt konkatencji

Czym jest łańcuch znaków

Ciąg znaków ułożony w pamięci

```
napis "Kot" w pamięci:
```

indeks:	0	1	2
znak:	'K'	'o'	't'
kod:	75	111	116

```
napis[0] -> 'K' # indeksowanie od zera
```

Typ prosty czy złożony?

Formalnie złożony – to ciąg znaków. Używany jednak jak typ prosty, dlatego omawiamy go tutaj.

Indeksowanie od zera

Pierwszy znak ma indeks 0, ostatni ma indeks długość – 1.

Reprezentacja w pamięci

Dwa podejścia

Zakończony zerem (język C)

```
'K' 'o' 't' '\0'
```

```
dlugosc = liczenie  
do znaku zerowego
```

Koszt odczytu długości

W C proporcjonalny do długości napisu. W Javie czy Pythonie natychmiastowy.

Z zapisaną długością (nowoczesne)

```
[3] 'K' 'o' 't'
```

```
dlugosc = odczyt  
jednej liczby
```

Znak zerowy w danych

W C napis nie może zawierać bajtu zerowego – obcina resztę zawartości.

Niemutowalność

Napisu nie da się zmienić w miejscu

```
>>> tekst = "Kot"
>>> tekst[0] = "B"
TypeError: 'str' object does not support item assignment

>>> tekst = "B" + tekst[1:]      # powstaje NOWY napis
>>> tekst
'Bot'
```

Niemutowalne

Python, Java, C#, JavaScript. Każda modyfikacja tworzy nowy obiekt.

Mutowalne

C, C++ (std::string), Ruby. Zawartość można zmienić w istniejącej pamięci.

Niemutowalność upraszcza pracę wielowątkową i pozwala bezpiecznie współdzielić napisy.

Podstawowe operacje

Zestaw, który trzeba znać

Operacja	Python	Java	Wynik dla "Ala ma kota"
Długość	<code>len(s)</code>	<code>s.length()</code>	11
Znak	<code>s[0]</code>	<code>s.charAt(0)</code>	'A'
Podciąg	<code>s[4:6]</code>	<code>s.substring(4,6)</code>	"ma"
Łączenie	<code>a + b</code>	<code>a + b</code>	—
Wyszukiwanie	<code>s.find("ma")</code>	<code>s.indexOf("ma")</code>	4
Zamiana	<code>s.replace(a,b)</code>	<code>s.replace(a,b)</code>	nowy napis

Wyszukiwanie zwraca indeks pierwszego wystąpienia albo wartość ujemną, gdy nie znaleziono.

Dzielenie i łączenie

Split i join

```
>>> dane = "Kowalski;Jan;18"
>>> pola = dane.split(";")
['Kowalski', 'Jan', '18']

>>> ";".join(pola)
'Kowalski;Jan;18'

>>> "  tekst  ".strip()      # usuwa białe znaki z brzegów
'tekst'
```

Praktyczne zastosowanie: wczytywanie plików CSV, parsowanie danych z formularza, przetwarzanie logów.

Koszt łączenia napisów

Pułapka wydajnościowa

```
# ZLE - kazde dodanie tworzy nowy napis
wynik = ""
for slowo in slowa:
    wynik += slowo           # 10 000 nowych obiektow

# DOBRZE
wynik = "".join(slowa)      # jedna alokacja
```

Python

"".join(lista)

Java / C#

StringBuilder

C++

std::string += jest tani

Dla kilku napisów różnicy nie widać. Przy tysiącach iteracji różnica bywa stukrotna.

Długość w znakach i w bajtach

To nie to samo

```
>>> tekst = "zazolc"          # 6 znakow ASCII
>>> len(tekst), len(tekst.encode('utf-8'))
(6, 6)

>>> tekst = "zazolc gesla"     # z polskimi znakami
>>> len(tekst), len(tekst.encode('utf-8'))
(12, 16)      # 4 znaki zajmuja po 2 bajty
```

Praktyczny skutek: pole VARCHAR(10) w bazie danych i limit 10 znaków w formularzu to nie zawsze ta sama granica. Emoji potrafi zająć cztery bajty.

Porównywanie i sortowanie

Porządek leksykograficzny

```
"Ala" < "Ola"      True      # bo 'A' (65) < 'O' (79)
"Zebra" < "ala"     True      # bo 'Z' (90) < 'a' (97)
```

porównanie znak po znaku, według kodów

```
"abc" == "abc"      True      # w Pythonie porównuje treść
s1.equals(s2)        # w Javie: == porównuje adresy!
```

Sortowanie po kodach nie jest sortowaniem alfabetycznym. Wielkie litery trafią przed małe, a polskie znaki na koniec. Do porządku alfabetycznego służy porównanie z regułami języka (collation).

Temat 5 – podsumowanie

Co trzeba umieć

- Łańcuch to ciąg znaków; indeksowanie zaczyna się od zera
- W C napis kończy się bajtem zerowym; w nowszych językach długość jest zapisana obok danych
- W Pythonie, Javie i C# napisy są niemutowalne – każda zmiana tworzy nowy obiekt
- Podstawowe operacje: długość, indeks, podciąg, łączenie, wyszukiwanie, zamiana, podział
- Łączenie napisów w pętli jest kosztowne – używamy join albo StringBuilder
- Długość w znakach różni się od długości w bajtach, gdy w tekście są znaki spoza ASCII
- Porównywanie napisów odbywa się według kodów znaków, nie według alfabetu

Ściąga – całość działu

Najważniejsze liczby

Zagadnienie	Wartość do zapamiętania
Liczba wartości na n bitach	2^n
Zakres bez znaku	0 do $2^n - 1$
Zakres ze znakiem (U2)	-2^{n-1} do $2^{n-1} - 1$
int8 / int16 / int32	-128..127 · ±32 767 · ±2,1 mld
Cyfry znaczące float / double	około 7 · około 15–16
Kody ASCII	'A' = 65 · 'a' = 97 · '0' = 48 · spacja = 32
ASCII / UTF-8	128 kodów na 7 bitach · 1 do 4 bajtów na znak
Kwoty pieniężne	typ dziesiętny albo grosze jako liczba całkowita

```
int float bool char str
```

Dziękuję za uwagę!

Ćwiczenia do działu: 02-typy-danych-cwiczenia.md

Opracował: Bartosz Bryniarski